Sealed Transforms: Secure, Verifiable Cloud Compute on User Data with Zero Knowledge

Tom Tang

March 10, 2026

Abstract

We present *sealed transforms*, a system for executing arbitrary computations on encrypted user data in the cloud such that the server learns nothing about the data, the intermediate values, or the encryption keys—while any third party can verify *what* computation was performed. The system uses WebAssembly (WASM) as a universal, deterministic execution format. Transforms are content-addressed WASM modules that operate inside a sandboxed runtime with no I/O, no system calls, and metered fuel. A three-layer composition model separates public logic (verifiable by anyone) from private key material (opaque to the server): Layer 1 modules implement reusable operations with published source; Layer 2 modules compose Layer 1 modules into pipelines; Layer 3 modules wrap any inner transform with user-specific encryption, embedding decryption and re-encryption keys in compiled WASM bytecode that the server executes but cannot inspect. The server sees ciphertext in and ciphertext out. Verification is achieved without revealing the wrapper's internals: the inner transform's content hash is declared as a WASM import, allowing anyone to audit the computation logic while the key material remains sealed. We formalize the security properties, prove that the system satisfies a zero-knowledge compute property under the WASM sandbox model, and analyze the trust assumptions across three deployment tiers: software isolation, trusted execution environments, and formal verification.

1 Introduction

Cloud computing requires users to surrender their data to a remote server. The server stores it, indexes it, transforms it, and returns results. At every step, the server has full access to plaintext. This is the fundamental bargain of cloud infrastructure: convenience in exchange for trust.

End-to-end encryption addresses storage—data at rest can be encrypted such that the server holds only ciphertext. But computation breaks the model. To sort, filter, aggregate, or transform data, the server must decrypt it, perform the operation, and re-encrypt the result. The moment computation touches plaintext, the server learns the data.

Three families of techniques address this gap. *Fully homomorphic encryption* (FHE) [3] permits computation on ciphertext without decryption, but incurs orders-of-magnitude overhead and supports only limited operation sets in practice. *Secure multi-party computation* (MPC) [6] distributes the computation across multiple non-colluding parties, but requires coordination infrastructure and degrades with latency. *Trusted execution environments* (TEEs) [2] provide hardware-enforced memory isolation, but depend on specific hardware and have suffered side-channel vulnerabilities [5].

This paper presents a fourth approach: *sealed transforms*. The key insight is that computation logic and encryption keys serve different roles and can be separated into independently auditable layers. A sealed transform is a WASM module that:

1. Decrypts the input using a key embedded in its bytecode.

2. Applies a publicly known, verifiable inner computation.
3. Re-encrypts the output under a different key embedded in its bytecode.

The server executes the WASM module in a sandboxed runtime with no I/O capabilities. It observes encrypted input, encrypted output, and the WASM bytecode itself—but compiled WASM is not trivially reversible, and the sandbox prevents the module from exfiltrating keys through side channels. The inner computation is publicly verifiable because it is a separate WASM module whose source code is published and whose content hash is declared as an import of the wrapper.

This paper makes three contributions:

1. A formal model of sealed transforms as composable, content-addressed WASM modules with a layered separation between public logic and private key material (§3).
2. A zero-knowledge compute property: the server learns nothing beyond what is deducible from the ciphertext lengths and the declared type signatures (§4).
3. A practical architecture with three security tiers—software sandbox, TEE-backed, and formally verified—with explicit trust assumptions for each (§5).

2 Problem Statement

A user stores data in the cloud and wants the cloud to compute on it—aggregate, transform, derive new values. The user also wants end-to-end encryption: the cloud should never see the plaintext. These two goals are in direct conflict. Computation requires access to data. Encryption denies access to data.

The standard resolution is to choose one: either the cloud computes on plaintext (sacrificing privacy) or the user computes locally (sacrificing the cloud's resources). Fully homomorphic encryption [3] offers a theoretical resolution but remains impractical for general computation.

We want a practical system where:

1. The user's data is encrypted at rest. The cloud stores ciphertext.
2. The cloud executes programs on the encrypted data and produces encrypted results.
3. The cloud learns nothing about the data, the intermediate values, or the encryption keys.
4. Anyone can verify *what program* the cloud executed, without seeing the data.

The system has two participants and one service:

- **User**: owns the data, holds the encryption keys, and decides what computations to run. The user encrypts data locally before uploading and decrypts results locally after downloading.
- **Cloud**: the user's server instance. Stores encrypted data, executes programs, returns encrypted results. The cloud follows the protocol but may attempt to learn data from what it handles. It sees ciphertext, WASM bytecode, and execution metadata (timing, memory allocation, gas consumption).
- **Platform**: publishes open-source transform logic, compiles it to WASM, and serves the compiled modules. The platform holds no user data and no keys. Its source code is public.

The fundamental challenge: the cloud must execute a program that internally decrypts data, computes on the plaintext, and re-encrypts the result—but the cloud must not be able to extract the decryption key from the program it is executing.

3 The Sealed Transform Model

3.1 WASM as the Execution Format

WebAssembly [4] is a portable, deterministic bytecode format with the following properties relevant to sealed computation:

- **Deterministic execution.** Given the same inputs and the same module, WASM produces the same outputs on every platform. Floating-point nondeterminism is eliminated via NaN canonicalization.
- **Sandboxed memory.** Each WASM module operates on its own linear memory. The host allocates and bounds this memory but does not share address space with the module during execution.
- **No ambient authority.** A WASM module has no capabilities unless the host explicitly provides them via imports. A module with no imports has no I/O—it is a pure function from memory to memory.
- **Fuel metering.** Modern WASM runtimes (wasmtime, wasmer) support fuel-based execution limits, guaranteeing termination within bounded cost.
- **Content addressability.** A WASM module is a byte sequence. Its SHA-256 hash serves as a globally unique, immutable identifier.

These properties make WASM suitable as a sealed execution format: a WASM module with no imports is a pure, deterministic, bounded computation that cannot exfiltrate data through side channels available to the module itself.

3.2 Module Interface

Every sealed transform exports a minimal interface:

```
(func (export "transform") (param i32 i32) (result i32))
(func (export "alloc")     (param i32)     (result i32))
(memory (export "memory") 1)
```

The host serializes the input `FieldValue` to bytes, allocates space in the module's memory via `alloc`, copies the serialized input, and calls `transform`. The module deserializes, computes, serializes the output into its own memory, and returns a pointer. The host reads the output and deserializes it back to a `FieldValue`.

Reversible transforms additionally export:

```
(func (export "inverse") (param i32 i32) (result i32))
```

Parameterized transforms (e.g., "multiply by 0.85") export an initialization function:

```
(func (export "init") (param i32 i32))
```

The parameter is included in the content hash: same logic with different parameters produces a different hash, and therefore a different transform identity.

3.3 Type System

Sealed transforms operate on a strictly typed value system. Types are declared at the schema level and enforced at transform registration.

Definition 1 (Field Types)**.** *The type system consists of scalar types, compound types, and an encryption wrapper:*

$$\begin{aligned} \mathit{ScalarType} &::= \texttt{String} \mid \texttt{Integer} \mid \texttt{Float} \mid \texttt{Boolean} \mid \texttt{Bytes} \\ \mathit{FieldType} &::= \texttt{Scalar}(s) \mid \texttt{Array}(s) \mid \texttt{Map}(s_1, s_2) \mid \texttt{Encrypted}(\mathit{FieldType}) \end{aligned}$$

where $s, s_1, s_2 \in \mathit{ScalarType}$.

The type system enforces two structural invariants:

Property 1 (Flatness)**.** *No type contains nested compound structures.* `Array` *and* `Map` *can only contain scalars.* `Array(Array(·))`*and*

Property 2 (Encryption Transparency)**.** `Encrypted(`T`)` *declares that the value is ciphertext whose plaintext conforms to type* T*. Double encryption* `Encrypted(Encrypted(·))` *is rejected at registration. A transform that accepts* `Encrypted(`T`)` *as input and produces* `Encrypted(`T'`)` *as output is understood to internally decrypt, compute, and re-encrypt—but the type signature alone reveals only that the plaintext types are* T *and* T'*.*

3.4 Three-Layer Composition

The central design principle is *separation of logic from keys.* This is achieved through a three-layer composition model.

3.4.1 Layer 1: Standard Transforms

A Layer 1 transform is a WASM module that implements a single, well-defined operation on plaintext values. Its source code is published. Anyone can compile the source and verify that the resulting WASM bytecode matches the registered content hash.

Transform	**Input Type**	**Output Type**	**Reversible**
`multiply`(c)	`Scalar(Float)`	`Scalar(Float)`	Yes
`divide`(c)	`Scalar(Float)`	`Scalar(Float)`	Yes
`add`(c)	`Scalar(Float)`	`Scalar(Float)`	Yes
`uppercase`	`Scalar(String)`	`Scalar(String)`	Yes
`lowercase`	`Scalar(String)`	`Scalar(String)`	Yes
`hash_sha256`	`Scalar(String)`	`Scalar(String)`	No
`array_sum`	`Array(Float)`	`Scalar(Float)`	No
`array_average`	`Array(Float)`	`Scalar(Float)`	No
`range_classify`	`Scalar(Integer)`	`Scalar(String)`	No

Table 1: Standard library transforms (Layer 1). Each is a separate WASM module with published source and a deterministic content hash.

Layer 1 transforms are the atoms of the system. They carry no secrets, make no assumptions about encryption, and are fully auditable.

3.4.2 Layer 2: Composed Transforms

A Layer 2 transform imports multiple Layer 1 (or other Layer 2) transforms and chains them into a pipeline. Its source code is published. Its imports are declared: anyone can trace the composition graph and verify that it applies exactly the stated sequence of operations.

Formally, a Layer 2 transform $P = T_n \circ T_{n-1} \circ \cdots \circ T_1$ has:

- Content hash $H(P)$ covering the full WASM bytecode.
- Import declarations $\{(T_i, H(T_i))\}$ for each constituent transform.
- Published source that anyone can compile and verify against $H(P)$.

Layer 2 transforms are pure logic pipelines. They still carry no secrets.

3.4.3 Layer 3: Sealed Wrappers

A Layer 3 transform wraps a Layer 1 or Layer 2 transform with encryption and decryption. It is compiled locally by the user, who embeds their encryption key into the WASM bytecode. The wrapper's source is not published. The compiled bytecode is opaque.

The critical property: the wrapper *declares its imports*. The WASM import table specifies the content hash of the inner transform. Anyone can look up that hash in the transform registry and inspect the inner transform's published source. The verification question changes from "what does this opaque module do?" to "does this module apply the transform it claims to import?"—and the WASM import mechanism guarantees that the module can only call the functions it declares.

Definition 2 (Sealed Wrapper)**.** *A sealed wrapper W for inner transform T is a WASM module such that:*

1. *W imports T by content hash $H(T)$.*
2. *W exports* `transform(ptr, len) -> ptr`*.*
3. *W's input type is* `Encrypted`*$(T.input_type)$.*
4. *W's output type is* `Encrypted`*$(T.output_type)$.*
5. *W's bytecode contains embedded key material that is not recoverable from the WASM binary under standard reverse-engineering assumptions (or provably irrecoverable under TEE deployment).*

3.5 Operational Flow

The three-layer model maps to a concrete operational cycle involving the platform, the user, and the user's cloud.

Step 1: The platform publishes transforms. Transform logic is developed as open-source code. The platform compiles each transform to a WASM module, registers it in the transform registry with its content hash, and serves the compiled module alongside any data views that reference it. Because the source is public, anyone can independently compile the source and verify that the resulting bytecode matches the registered hash. The platform acts as a compiler and distributor of verifiable computation—it does not hold user data or keys.

Step 2: The user wraps a transform. The user selects a platform-published transform (e.g., `multiply_085`, hash $H(T)$). Locally, the user compiles a Layer 3 wrapper that imports T and embeds their encryption key k. The user registers the resulting WASM module in the transform registry of their own cloud instance. The wrapper's content hash $H(W)$ covers everything—the import binding, the embedded key, and the wrapper logic.

Step 3: The user's cloud executes the sealed transform. When a query requires the derived value, the cloud loads W into the WASM runtime, resolves the import of T from the registry, passes the encrypted input into W's sandbox, and receives the encrypted output. The cloud never sees plaintext. It sees $\mathrm{Enc}(x)$ in, $\mathrm{Enc}(y)$ out, and the WASM bytecode of W.

Step 4: The user reads the result. The user's client receives the encrypted output and decrypts it locally with key k.

The critical invariant is that the platform's published transforms are the *only* computation that touches plaintext—and they are fully public and auditable. The wrapper adds only the encryption layer. The cloud executes both but sees neither the plaintext nor the keys.

3.6 Content Addressing and the Transform Registry

Every transform—Layer 1, 2, or 3—is identified by the SHA-256 hash of its WASM bytecode. The transform registry maps hashes to metadata:

- **Content hash**: the transform's identity.
- **Input/output types**: declared `FieldType` signatures.
- **Imports**: content hashes of all imported transforms.
- **Reversibility**: whether the module exports `inverse`.
- **Gas limit**: maximum fuel for execution.
- **Security label constraint**: minimum output label ($\ell_{\text{in}} \sqsubseteq \ell_{\text{out}}$).

Registration enforces:

1. **Type compatibility**: the declared input type matches the source field's type; the declared output type matches the target field's type.
2. **Import resolution**: every imported transform must already exist in the registry.
3. **Acyclicity**: the import graph must be a DAG.
4. **Encryption consistency**: if the source field is `Encrypted`(T), the transform's input must be `Encrypted`(T).
5. **No double encryption**: `Encrypted(Encrypted(·))` types are rejected.
6. **Gas bounds**: the declared gas limit must fall within system-configured bounds.
7. **Reversibility consistency**: if declared reversible, the module must export `inverse`.

4 Security Analysis

4.1 Threat Model

The adversary is the cloud operator: an honest-but-curious party that executes the protocol faithfully but attempts to learn the plaintext data, the encryption keys, or any intermediate values. The adversary has:

- Full access to all ciphertext (encrypted field values at rest).
- Full access to all registered WASM bytecode (including Layer 3 wrappers).
- The ability to execute WASM modules and observe their behavior (timing, memory allocation patterns).
- *No* access to WASM linear memory during execution (enforced by the runtime sandbox).
- *No* ability to modify WASM bytecode without changing the content hash.

4.2 Zero-Knowledge Compute

Theorem 1 (Zero-Knowledge Compute under Sandbox Model)**.** *Let W be a Layer 3 sealed wrapper that imports inner transform T. Let x be the plaintext input and $y = T(x)$ be the plaintext output. The cloud executes W on $\mathrm{Enc}_k(x)$ and receives $\mathrm{Enc}_k(y)$. Under the following assumptions:*

1. *The encryption scheme is IND-CPA secure.*
2. *The WASM sandbox provides memory isolation (the host cannot read W's linear memory during execution).*
3. *W has no imports other than T (no I/O, no host functions that leak data).*

The cloud's view $\mathcal{V} = (\mathrm{Enc}_k(x),\ \mathrm{Enc}_k(y),\ W,\ H(T))$ is computationally indistinguishable from a simulation that knows only $|x|$, $|y|$, W, and $H(T)$.

Proof sketch. By the IND-CPA security of the encryption scheme, $\mathrm{Enc}_k(x)$ reveals nothing about x beyond $|x|$, and $\mathrm{Enc}_k(y)$ reveals nothing about y beyond $|y|$. The WASM module W is a static byte sequence—it does not change between executions and is known to the cloud regardless of the input. The memory isolation property ensures that the cloud cannot observe decrypted values, intermediate computations, or key material within W's linear memory during execution. Since W has no I/O imports, it cannot transmit information to the host except through its return value, which is $\mathrm{Enc}_k(y)$. The content hash $H(T)$ is public metadata. Therefore, a simulator given only $|x|$, $|y|$, W, and $H(T)$ can produce a view indistinguishable from $\mathcal{V}$ by generating random ciphertexts of the appropriate lengths. □

Caveats. The proof assumes ideal memory isolation. In practice, the WASM runtime executes on the server's hardware, and the server may observe:

- **Timing.** Execution time may depend on the input (e.g., branch-dependent control flow). Constant-time WASM implementations mitigate this.
- **Memory access patterns.** If the host can observe which memory pages the WASM module accesses (but not the values), this may leak information about the computation structure. ORAM techniques can mitigate this at significant performance cost.

- **Fuel consumption.** The amount of gas consumed may correlate with input size or values.

These side channels are addressed by the tiered deployment model (§5).

4.3 Verifiable Computation

Theorem 2 (Verifiability)**.** *Let W be a Layer 3 sealed wrapper with declared import T and content hash $H(T)$. Let S_T be the published source code of T. If* $\text{compile}(S_T) = B_T$ *and* $\text{SHA256}(B_T) = H(T)$*, then any verifier can confirm that W applies T to the decrypted input, without knowledge of W's internal key material.*

Proof. The WASM import mechanism is binding: W declares that it imports a function from module T identified by $H(T)$. The WASM runtime resolves this import at instantiation time by linking the module whose bytecode hashes to $H(T)$. W cannot call any function other than those declared in its import table—this is enforced by the WASM type system and validation algorithm [4]. Therefore, the only computation T that W can invoke on the decrypted data is the one identified by $H(T)$. The verifier compiles S_T, verifies $H(T)$, and reads S_T to understand the computation. □

The verifier does *not* need to read W's bytecode, understand its key management, or trust its author. The verification reduces to: (1) the inner transform T does what its source says, and (2) W declares that it imports T. The WASM runtime enforces the binding.

4.4 Key Concealment

The security of Layer 3 wrappers depends on the difficulty of extracting embedded keys from compiled WASM bytecode.

Property 3 (Key Concealment)**.** *Let W be a Layer 3 wrapper containing an embedded key k. The difficulty of extracting k from W's bytecode depends on the deployment tier:*

1. ***Software sandbox*** *(Tier 1): extraction requires reverse-engineering compiled WASM. Obfuscation techniques (white-box cryptography, control-flow flattening) increase difficulty but do not provide provable security. This tier is appropriate for commercial sensitivity, not for adversaries with nation-state capabilities.*

2. ***TEE deployment*** *(Tier 2): the WASM module executes inside a hardware enclave (Intel SGX, AWS Nitro Enclaves). The cloud cannot read enclave memory. Key extraction is infeasible absent hardware vulnerabilities. Remote attestation proves that the enclave is executing the correct WASM module.*

3. ***Formal verification*** *(Tier 3): the WASM module is accompanied by a formal proof (e.g., in Coq or Lean) that its execution does not write key material to its output buffer. This provides mathematical certainty independent of hardware trust assumptions.*

5 Deployment Tiers

The system supports three deployment tiers with progressively stronger trust assumptions. All three use the same WASM modules, the same registry, and the same type system. The tiers differ only in the execution environment and the resulting security guarantees.

5.1 Tier 1: Software Sandbox

The WASM module executes in a standard WASM runtime (wasmtime) with:

- No WASI capabilities (no filesystem, no network, no clock).
- Fuel metering (bounded execution).
- Memory isolation (the host uses the WASM API, not direct memory access).

Trust assumption. The WASM runtime correctly implements memory isolation and the import mechanism. The server does not attempt to extract keys from WASM bytecode through reverse engineering.

Appropriate for. Commercial data sharing where the server operator is a reputable cloud provider with contractual obligations. Insider threats are addressed by policy and audit, not cryptographic proof.

5.2 Tier 2: Trusted Execution Environment

The WASM runtime itself executes inside a TEE (Intel SGX, AMD SEV, AWS Nitro Enclaves). The server operator cannot read enclave memory, even with root access.

Trust assumption. The TEE hardware is not compromised. Remote attestation correctly identifies the WASM runtime and the WASM module being executed.

Additional guarantees over Tier 1.

- Key material in the WASM module's linear memory is encrypted by the TEE's memory encryption engine.
- The server cannot attach a debugger or inject code into the enclave.
- Remote attestation provides a cryptographic proof chain: the client can verify that a specific WASM module (identified by content hash) is executing inside a specific enclave (identified by attestation report).

Appropriate for. Regulated industries (healthcare, finance) where compliance requires cryptographic proof that the server cannot access plaintext. Suitable for data subject to GDPR, HIPAA, or similar regulations.

5.3 Tier 3: Formally Verified Execution

The WASM module is accompanied by a machine-checkable proof that:

1. The module's `transform` function writes only encrypted data to the output buffer.
2. Key material is used only as arguments to the encryption and decryption functions.
3. No key material appears in the return value under any input.

Trust assumption. The proof checker (Coq, Lean) is correct. The WASM operational semantics used in the proof are faithful to the runtime implementation.

Appropriate for. High-assurance scenarios where neither the server nor the hardware is fully trusted. Provides mathematical certainty that exceeds both software and hardware guarantees.

6 Data Flow

6.1 Write Path

The user encrypts locally before uploading to the cloud:

1. User holds plaintext value v and field encryption key k.
2. User computes $c = \mathrm{Enc}(v, k)$.
3. User sends $(c, \texttt{Encrypted}(T))$ to the cloud, where T is the plaintext type.
4. Cloud stores c alongside the type declaration. Cloud sees ciphertext and type metadata only.

6.2 Transform Path

The cloud executes a sealed wrapper on encrypted data:

1. Cloud retrieves encrypted input $c_{\text{in}} = \mathrm{Enc}_k(x)$ from storage.
2. Cloud loads Layer 3 wrapper W (identified by content hash) into the WASM runtime.
3. Cloud resolves W's import of inner transform T (identified by $H(T)$) from the registry.
4. Cloud serializes c_{in}, copies it into W's linear memory, and calls `transform`.
5. Inside W: decrypt c_{in} using embedded k; apply T to plaintext; encrypt result using embedded k; write c_{out} to output buffer.
6. Cloud reads c_{out} from W's memory. Cloud sees ciphertext only.
7. Cloud stores c_{out} in the derived field.

6.3 Read Path

The user decrypts locally:

1. Cloud returns $c_{\text{out}} = \mathrm{Enc}_k(y)$ to the user's client.
2. Client decrypts: $y = \mathrm{Dec}(c_{\text{out}}, k)$.

At no point does the cloud observe x, y, or k.

7 Verification Protocol

A third-party verifier (e.g., a regulator or auditor) wants to confirm what computation was applied to the user's data, without seeing any data.

1. Verifier obtains the content hash of the Layer 3 wrapper W from the fold's metadata.
2. Verifier reads W's import declarations from the registry: inner transform T with hash $H(T)$.
3. Verifier obtains the published source code S_T of transform T.
4. Verifier compiles S_T and confirms $\text{SHA256}(\text{compile}(S_T)) = H(T)$.
5. Verifier reads S_T to understand the computation (e.g., "multiply salary by 0.85").
6. Verifier reads W's declared input/output types: `Encrypted(Scalar(Float))` $\rightarrow$ `Encrypted(Scalar(Float))`
7. Verifier concludes: W takes an encrypted float, applies the published multiplication transform, and produces an encrypted float. The verifier does not need to inspect W's bytecode or know any keys.

Under Tier 2 (TEE), the verifier additionally obtains a remote attestation report confirming that the cloud's enclave is running the exact WASM module identified by $H(W)$.

8 Key Management

8.1 Key Embedding

In the sealed wrapper model, the user's encryption key is embedded in the WASM bytecode at compile time. The user generates a wrapper WASM locally that contains:

- k: the user's encryption key for the field.

The wrapper is compiled locally by the user and registered in the transform registry of their cloud instance. The user is the only party that possesses the source code of the wrapper.

8.2 Key Rotation

When a key is rotated, all wrappers embedding the old key must be re-compiled:

1. User generates a new key k'.
2. User re-encrypts stored data: $c' = \text{Enc}(v, k')$ for all affected fields.
3. User compiles new wrappers W' embedding k' and registers them. Each new wrapper has a new content hash.
4. Fold definitions are updated to reference the new wrapper hashes.
5. Old wrappers are retained in the registry (content-addressed, immutable) but are no longer referenced by any active fold.

	FHE	**MPC**	**TEE-only**	**Sealed Transforms**
Arbitrary computation	Limited†	Yes	Yes	Yes
Server sees plaintext	No	No	No‡	No
Verifiable logic	No*	No*	Via attestation	Yes (imports)
Single-server deployment	Yes	No	Yes	Yes
Standard hardware	Yes	Yes	No	Yes (Tier 1)
Performance overhead	10^3–$10^6\times$	10–$10^3\times$	1–2$\times$	1–5$\times$
Key management	Client-side	Distributed	Enclave-sealed	Bytecode-embedded

Table 2: Comparison of approaches to computation on encrypted data. †: FHE supports arbitrary circuits in theory but is impractical for complex operations. ‡: TEE-only requires hardware trust. $*$: FHE and MPC verify correctness of the protocol, not the logic being computed.

9 Comparison with Related Work

Fully Homomorphic Encryption [3]. FHE computes on ciphertext without decryption, providing stronger confidentiality guarantees than software-sandbox sealed transforms. However, FHE's computational overhead (10^3–$10^6\times$ for general circuits) makes it impractical for interactive database queries. Sealed transforms achieve comparable confidentiality at near-native speed when deployed in a TEE.

Secure Multi-Party Computation [6]. MPC distributes trust across multiple non-colluding parties, eliminating single-point compromise. Sealed transforms operate on the user's own cloud instance, making deployment simpler but requiring trust in the execution environment. MPC addresses a different problem (multi-party collaboration without a trusted coordinator); sealed transforms address single-user cloud compute without trusting the cloud.

Trusted Execution Environments [2]. TEE-only approaches (e.g., running arbitrary code in SGX enclaves) provide hardware-backed isolation but lack the layered verifiability of sealed transforms. A TEE running an opaque binary proves *that* a specific binary ran, but not *what* that binary does. Sealed transforms add the import-based verification layer: the inner logic is public and auditable, even when the wrapper is opaque.

White-Box Cryptography [1]. White-box crypto aims to protect keys embedded in software executing in an untrusted environment—precisely the Tier 1 scenario. Sealed transforms adopt this goal but do not rely on white-box techniques alone; Tier 2 and Tier 3 provide stronger guarantees.

10 Limitations and Future Work

Side channels. Tier 1 deployment is vulnerable to timing and memory-access-pattern side channels. Constant-time WASM compilation and ORAM-based memory access are active research areas.

WASM binary analysis. Decompilation of WASM bytecode is an active field. As tooling improves, Tier 1 key concealment weakens. This reinforces the importance of Tier 2 and Tier 3 deployments for high-sensitivity data.

Gas-based information leakage. The amount of fuel consumed by a transform may correlate with input characteristics. Padding execution to a fixed gas cost would eliminate this channel at the expense of performance.

Multi-input transforms. The current model supports single-input transforms. Extending to multi-input sealed transforms (e.g., joins across two encrypted fields) requires careful key management and is left for future work.

Composing Layer 3 wrappers. A wrapper that imports another wrapper creates a chain of trust. The verification model extends naturally (each import is a declared hash), but key management complexity grows. Formal bounds on composition depth merit investigation.

Standardized wrapper tooling. Currently, users compile wrappers locally using general-purpose WASM toolchains. A dedicated tool that takes a transform hash and key material as input and produces a sealed wrapper WASM as output would simplify adoption.

11 Conclusion

Sealed transforms provide a practical path to zero-knowledge cloud compute. By separating computation logic (public, auditable WASM modules) from encryption key material (embedded in opaque wrapper bytecode), the system achieves two properties simultaneously: the cloud executes the computation without learning the data, and any third party can verify what computation was performed without accessing the data.

The three-layer composition model ensures that the verification burden falls on the inner transforms—which are small, published, and auditable—rather than on the wrappers, which are opaque by design. The WASM import mechanism provides a binding guarantee: a wrapper can only invoke the transforms it declares, and those transforms are identified by content hash.

Three deployment tiers provide a spectrum of trust assumptions: software sandboxing for commercial deployments, TEE-backed execution for regulatory compliance, and formal verification for high-assurance scenarios. The same WASM modules, the same registry, and the same type system serve all three tiers. The choice of tier is a deployment decision, not an architectural one.

References

[1] Stanley Chow, Philip Eisen, Harold Johnson, and Paul C. Van Oorschot. White-box cryptography and an AES implementation. In *Selected Areas in Cryptography (SAC)*, pages 250–270, 2002.

[2] Victor Costan and Srinivas Devadas. Intel SGX explained. Technical Report 2016/086, IACR ePrint Archive, 2016.

[3] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing (STOC)*, pages 169–178, 2009.

[4] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. Bringing the web up to speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 185–200, 2017.

[5] Jo Van Bulck, Marina Minkin, Ofir Weisse, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F. Wenisch, Yuval Yarom, and Raoul Strackx. Foreshadow: Extracting the keys to the Intel SGX kingdom with transient out-of-order execution. In *Proceedings of the 27th USENIX Security Symposium*, pages 991–1008, 2018.

[6] Andrew Chi-Chih Yao. How to generate and exchange secrets. In *Proceedings of the 27th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 162–167, 1986.